Final Assignment

Verdan Cassandra

Schedule search

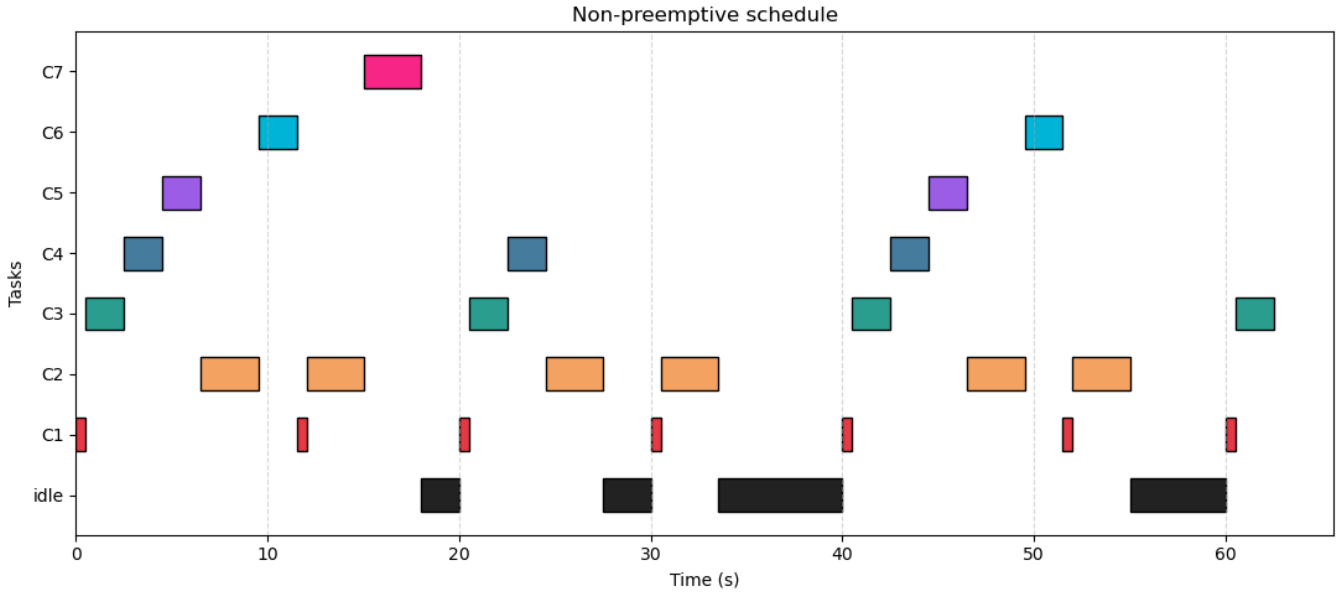

Figure 1: Gantt schedule of the system

SOMMAIRE

Introduction . . . 1

I Execution time of task τ_1 **2**
- I.1 Multiplication code in C . . . 2
  - I.1.1 Execution Time Analysis . . . 3
- I.2 Python code for execution time measurement . . . 3
  - I.2.1 Python code for execution time measurement . . . 3

II Schedulability analysis **5**
- II.1 Case 1: No deadline miss allowed . . . 6
  - II.1.1 Schedule representation . . . 6
- II.2 Case 2: Task τ_5 is allowed to miss a deadline . . . 6

III Response time **7**

Introduction

In this project, we focus on the analysis of a set of periodic tasks in order to determine if they can be correctly scheduled on a processor. The main objective is to verify that no deadlines are missed while also minimizing the total waiting time of the system. To do this, we first estimate the execution time of a task using a C program, then we analyze the results with Python. Finally, we simulate different scheduling orders to find the most efficient one under a non-preemptive policy.
More generally, this work is part of the study of real-time embedded systems. In such systems, each task must be executed within specific timing constraints. A task is typically defined by its Worst Case Execution Time (WCET), its period, and its deadline. These parameters allow us to model the system and evaluate its behavior.
Scheduling is therefore a key aspect, as it defines how processor time is distributed between tasks. A correct scheduling strategy ensures that all tasks meet their deadlines and that the system operates reliably. In this project, we use these concepts to study the schedulability of a given task set and to identify an optimal execution order.

I Execution time of task τ_1

The first step of the project is to determine the execution time of task τ_1. This task is based on a simple multiplication operation between two large randomly generated numbers. Since one multiplication alone would be too fast to measure correctly, the operation is repeated many times inside the C program. The objective is not to use the numerical result itself, but to create a measurable execution time that can then be analyzed statistically.

I.1 Multiplication code in C

The C program contains a function called `product`. This function takes two integers as inputs and returns their product as a `long long`. This type is used because the multiplication of two large numbers can easily exceed the range of a normal integer.
In the main function, the random generator is initialized with `srand`. Then, the program repeats the multiplication 10 000 000 times. At each iteration, two large random numbers are generated, and their product is added to the variable `result`. This final value is printed only to make sure that the compiler really executes the calculations and does not remove them during optimization.

```c
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

long long product(int x, int y)
{    return (long long)x * y;
}
int main(void)
{    srand((unsigned int)time(NULL));
    const int N = 10000000;
    long long result = 0;

    for (int i = 0; i < N; i++)
    {   int x = rand() % 10000000 + 100000;
        int y = rand() % 10000000 + 100000;
        result += product(x, y);
    }

    printf("%lld\n", result);
    return 0;
}
```

Code I.1: C code used to execute the multiplication task

We compile the program in WSL using the following command:

```
gcc multiplication.c -O2 -o product
```

Then, the executable was launched with:

```
./product
```

The value printed in the terminal corresponds to the accumulated result of all the multiplications. It is not used directly in the schedulability study, but it confirms that the calculation loop has been executed correctly.

```
angl_cass@cass-vdn:/mnt/c/Users/cassi/Real_Time_information_Systems/Real-Ti
me-Info-system-_Project_Cassandra$ gcc multiplication.c -O2 -o product
angl_cass@cass-vdn:/mnt/c/Users/cassi/Real_Time_information_Systems/Real-Ti
me-Info-system-_Project_Cassandra$ ./product
1449188169483861819
```

Figure I.1: Execution of the multiplication program in WSL

Just to check if each time wxe had coherent results, we did the test multiple time.

I.1.1 Execution Time Analysis

To evaluate the execution time, the program was executed repeatedly, and the execution time of each run was measured and recorded in order to build a dataset for us to analyse.

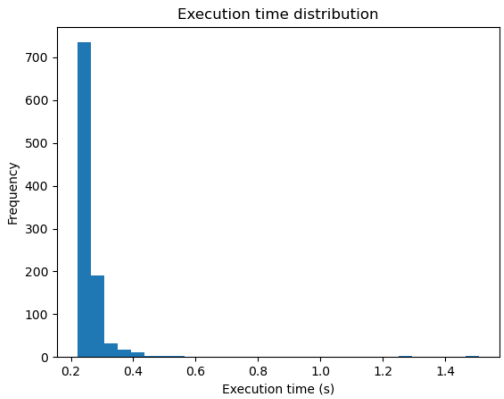

Figure I.2: Execution time distribution of the program

The histogram represents the distribution of execution times obtained over multiple runs of the program. Most executions are concentrated around a narrow interval, approximately between 0.23 s and 0.27 s, which indicates that the program behaves consistently under normal conditions. However, a small number of executions take significantly longer, creating a long tail in the distribution. These higher values are due to external factors, but this variability highlights the importance of considering worst-case scenarios when evaluating execution time.

I.2 Python code for execution time measurement

I.2.1 Python code for execution time measurement

After validating the C program, we make a Pytrhc=on program to measure the execution time of the task. The idea is to run the executable multiple times in order to obtain a representative set of execution times.
In this project, the program is executed 1000 times. For each iteration, the execution time is measured and stored in a list. This allows us to analyze the distribution of execution times instead of relying on a single value.

Once all executions are completed, we compute the minimum value, the first quartile, the median, the third quartile, and the maximum. Theygive a better understanding of the typical behavior of the task as well as the variability of the execution time.
However, the maximum value alone is not always reliable, as it may include occasional delays caused by the operating system or the execution environmene, so we use the filtered maximum with a safety factor to obtain a more robust estimation of the WCET.

```
import subprocess
import numpy as np
import matplotlib.pyplot as plt

NB_RUNS = 1000
times = []

for i in range(NB_RUNS):
    if i % 50 == 0:
```

```python
        print("Run", i, "/", NB_RUNS)

    result = subprocess.run(
        ["/usr/bin/time", "-f", "%e", "./product"],
        stderr=subprocess.PIPE,
        stdout=subprocess.DEVNULL,
        text=True
    )

    try:
        t = float(result.stderr.strip())
        times.append(t)
    except ValueError:
        continue

times = np.array(times)

minimum = np.min(times)
q1 = np.percentile(times, 25)
median = np.percentile(times, 50)
q3 = np.percentile(times, 75)
maximum = np.max(times)

raw_wcet = maximum

iqr = q3 - q1
upper_limit = q3 + 1.5 * iqr
filtered_times = times[times <= upper_limit]

max_filtered = np.max(filtered_times)
robust_wcet = max_filtered * 1.2

print("Min :", minimum)
print("Q1 :", q1)
print("Median :", median)
print("Q3 :", q3)
print("Max :", maximum)
print("Raw WCET :", raw_wcet)
print("Filtered max :", max_filtered)
print("Robust WCET :", robust_wcet)

plt.hist(times, bins=30)
plt.xlabel("Execution time (s)")
plt.ylabel("Frequency")
plt.title("Execution time distribution")
plt.show()
```

Code I.2: Python script used to measure execution times

The results show that most execution times are concentrated around a stable value, while a few higher values appear due to external factors. By filtering these outliers and applying a safety factor, a more realistic WCET is obtained for the task.

```
Run 750 / 1000
Run 800 / 1000
Run 850 / 1000
Run 900 / 1000
Run 950 / 1000
Min : 0.25
Q1 : 0.26
Median : 0.28500000000000003
Q3 : 0.33
Max : 0.74
Raw WCET : 0.74
Filtered max : 0.43
Robust WCET : 0.516
```

Figure I.3: Distribution of execution times for task τ_1

II

Schedulability analysis

After determining the execution time of task τ_1, we can now analyze the schedulability of the complete task set. The objective is to verify whether all tasks can be executed without missing their deadlines, and to identify an execution order that minimizes the total waiting time.
The system is composed of 7 periodic tasks. Each task is defined by its computation time C_i and its period T_i. The deadlines are assumed to be equal to the periods. The value of C_1 is taken from the WCET estimation obtained previously.

Task	C_i (s)	T_i (s)
τ_1	0.516	10
τ_2	3	10
τ_3	2	20
τ_4	2	20
τ_5	2	40
τ_6	2	40
τ_7	3	80

Tableau II.1: Task set

To verify if the system is schedulable, we first compute the processor utilization:

$$U = \sum_{i=1}^{7} \frac{C_i}{T_i}$$

$$U = \frac{0.516}{10} + \frac{3}{10} + \frac{2}{20} + \frac{2}{20} + \frac{2}{40} + \frac{2}{40} + \frac{3}{80} \approx 0.79$$

Since $U < 1$, the processor has enough capacity to execute all tasks. However, this condition alone is not sufficient in the non-preemptive case, which is why a full simulation is required.
The analysis is performed over the hyperperiod of the system, which corresponds to the least common multiple of all task periods:

$$H = \text{LCM}(10, 20, 40, 80) = 80 \text{ s}$$

This means that the schedule repeats every 80 seconds, so it is sufficient to study the system over this interval.
In the simulation, two main assumptions are made. First, only one task can be executed at a given time. Second, tasks are independent and do not share resources.
To explore all possible scheduling configurations, a Python script is used. The script generates all possible priority orders between the tasks using permutations. For 7 tasks, this results in:

$$7! = 5040 \text{ possible priority orders}$$

Each configuration is then simulated over the hyperperiod to check if deadlines are respected. For valid schedules, the total waiting time is computed. The complexity of this approach is therefore:

$$O(H \times n!)$$

which quickly becomes expensive as the number of tasks increases.

II.1 Case 1: No deadline miss allowed

In this first case, we impose that all tasks must meet their deadlines. To study this, we use our Python script to test all possible priority orders and keep only the valid ones. We run a loop over all 5040 possible permutations of the tasks. For each configuration, we simulate the execution over the hyperperiod in a non-preemptive context, meaning that once a task starts, we let it finish before switching to another one.
For each valid schedule, we compute the total waiting time. This corresponds to the sum of all delays between the release of each task and the moment it actually starts executing. Once we have these values, we sort the valid schedules based on this waiting time to find the most efficient ones.
Some configurations lead to the same minimal waiting time, so we keep all of them. Then, we select one of these optimal schedules to visualize the result.

- Number of schedulable sets: 1440
- Total waiting time: 106.256 s
- Number of optimal schedules: 4

II.1.1 Schedule representation

To better visualize the execution of the tasks, we makje a Gantt chart from the simulation results because it shows the order in which the tasks are executed over time, as well as the idle periods of the processor.

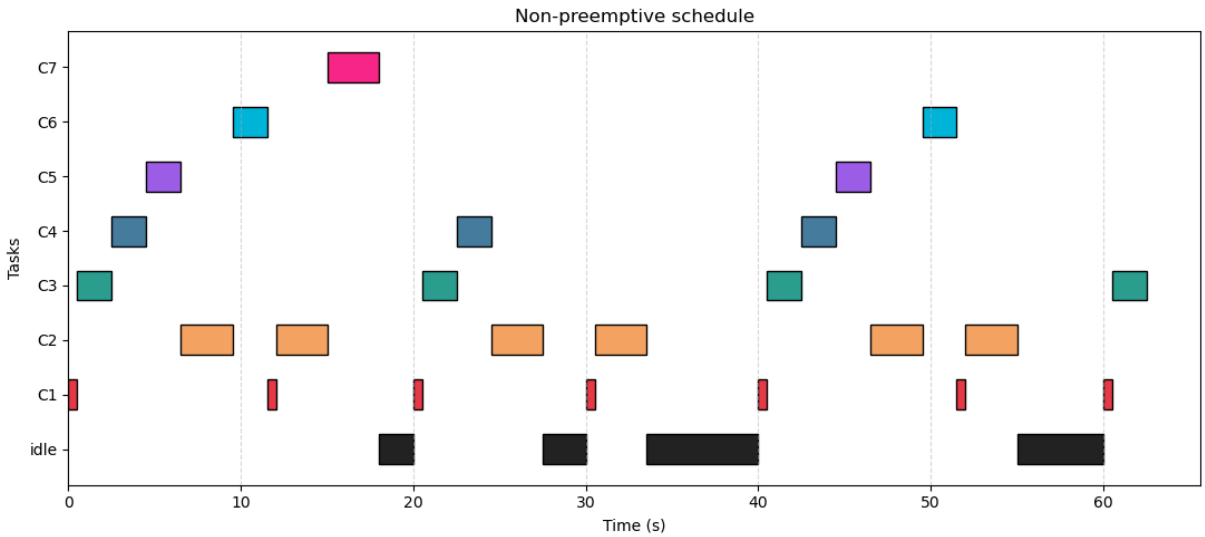

Figure II.1: Non-preemptive schedule for the selected priority order

Each task is associated with a specific color, which makes it easier to distinguish between them. The horizontal axis represents time, while the vertical axis corresponds to the different tasks. When a task is executed, a colored block appears on the chart at the corresponding time interval.
This visualization helps to understand how the scheduling policy affects the system behavior. In particular, it allows us to observe the execution order, the waiting times, and the idle periods of the processor.

II.2 Case 2: Task τ_5 is allowed to miss a deadline

In this second case, τ_5 is allowed to miss a deadline, while all other tasks must still meet their timing constraints, so we use the same code. During the simulation, if a deadline is missed by τ_5, the schedule is still considered valid, while any deadline miss from the other tasks leads to its rejection. Apart from this condition, the methodology remains unchanged. After running the simulations, the results obtained are identical to those of Case 1. The number of valid schedules, as well as the configurations minimizing the total waiting time, remain the same. This suggests that task τ_5 does not play a critical role in the schedulability of the system. Allowing it to miss a deadline does not improve the feasibility of the schedules nor the optimal solutions. Therefore, τ_5 does not significantly influence the overall behavior of the system.

III

Response time

The response time corresponds to the delay between the moment a task is released and the moment it finishes its execution. In other words, it includes both the execution time and the waiting time before the task actually starts.

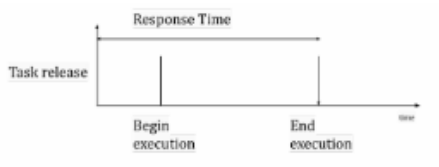

Figure III.1: Response time illustration

To compute it, we reuse the same simulation loop as in the schedulability analysis. While we simulate each schedule, we track the response time for every job. At the end, for each task, we keep the maximum value observed, since we are interested in the worst-case behavior.
The results obtained with our simulation are the following:

$$C_1 = 2.03 \text{ s} \quad C_2 = 9.52 \text{ s}$$
$$C_3 = 2.52 \text{ s} \quad C_4 = 4.52 \text{ s}$$
$$C_5 = 6.52 \text{ s} \quad C_6 = 11.52 \text{ s}$$
$$C_7 = 18.03 \text{ s}$$

We can see that the response time is always higher than the execution time. This is expected, since each task may have to wait before being executed depending on the current schedule. We also observe that tasks with higher priority tend to have lower response times, while lower priority tasks accumulate more delay. For example, task C1 has a response time close to its WCET, while task C7 has a much higher value. This confirms that the priority order has a direct impact on the system performance. Even if all deadlines are respected, the response time can vary significantly depending on how the tasks are scheduled.

Conclusion

In this project, we studied the schedulability of a real-time system composed of seven periodic tasks under a non-preemptive policy. We first estimated the execution time of a task using a C program and obtained a reliable WCET value through statistical analysis.
Then, we evaluated all possible priority orders over the hyperperiod to identify the valid schedules and select the one minimizing the total waiting time. The results show that the system is schedulable and that several optimal configurations exist.

Allowing task C5 to miss a deadline did not change the results, which suggests that it is not critical to the system behavior. Overall, this study confirms that the choice of priorities has a direct impact on system performance, even when all deadlines are respected.